

mode; 4000 values are added to this file in every *filegen()* call. Then the *tempc.txt* file is hashed, and this hash stored in *hashes.txt*. So for every single call of *func2()*, three calls of *filegen()* are made—3 MD5 hashes are computed for every increment of *p*. MD5 hashing is used to keep the CPU busy. *func3()* comprises of the 'l' variable, which is incremented to 50000, and will increment *counter3* every time to track the calls. Every function has different time consumptions.

```
/*
Name :Subodh Pachghare
Author :I am the author!!!
Description :Function calling test program for Gprof
*/
#include <stdio.h>
int counter1=0; /*Global Variable*/
int counter2=0; /*Global Variable*/
int counter3=0; /*Global Variable*/
int main()
{
int p=0;
system("> /root/hashes.txt"); /*File will be overwritten or
Created*/
system("> /root/tempc.txt"); /*File will be overwritten or
Created*/
printf("Initial value in Main function= %d\n",p);
while(p<10000) /*Condition for while loop*/
{
if(p>9600 && p<10000)
{
func1();
}
else
{
func3();
}
p++;
}
printf("Last value in Main function (value(p) incremented for)
= %d\n",p);
printf("Func1(),Func2(),Func3() & filegen() called for %d %d %d
times\n",counter1,counter1,counter3,counter2);
}
func1()
{
int j=0;
while(j<=66667) /*Another operation to eat CPU
cycles*/
j++;
if(j==66668)
{
counter1++;
}
```

```
func2(); /*Call to Function func2()*/
}
func2()
{
int m=0;
for (m=1;m<1000000;m++)
{
if(m<4)
{
filegen(); /*Call to Function filegen()*/
counter2++;
}
else
{
return 0; /*return to the func1()*/
}
}
}
filegen()
{
FILE *fo,*fg;
int f=0;
fo=fopen("/root/tempc.txt","a");
while(f<4000)
{
fprintf(fo,"%d\n",f);
f++;
}
fclose(fo);
system("md5sum /root/tempc.txt >> /root/hashes.txt");
/*Unique MD5 hash generation*/
}
func3()
{
int l=0,b=0;
while(l<=50000)
{
b=l;
l++;
}
if(l==50001)
{
counter3++;
}
}
```

Killing code before completion: Note that this code cannot be interrupted with Ctrl-C due to presence of system call. So if you want to stop execution in between, use *kill -9 'pidof a.out'* from another terminal to send a SIGKILL signal.

Reporting CPU activity

To report CPU utilisation in user-space and kernel-space